

O'REILLY®

Designing Machine Learning Systems

An Iterative Process
for Production-Ready
Applications



Chip Huyen

Praise for *Designing Machine Learning Systems*

There is so much information one needs to know to be an effective machine learning engineer. It's hard to cut through the chaff to get the most relevant information, but Chip has done that admirably with this book. If you are serious about ML in production, and care about how to design and implement ML systems end to end, this book is essential.

—Laurence Moroney, AI and ML Lead, Google

One of the best resources that focuses on the first principles behind designing ML systems for production. A must-read to navigate the ephemeral landscape of tooling and platform options.

—Goku Mohandas, Founder of Made With ML

Chip's manual is the book we deserve and the one we need right now. In a blooming but chaotic ecosystem, this principled view on end-to-end ML is both your map and your compass: a must-read for practitioners inside and outside of Big Tech—especially those working at “reasonable scale.” This book will also appeal to data leaders looking for best practices on how to deploy, manage, and monitor systems in the wild.

—Jacopo Tagliabue, Director of AI, Coveo; Adj. Professor of MLSys, NYU

This is, simply, the very best book you can read about how to build, deploy, and scale machine learning models at a company for maximum impact. Chip is a masterful teacher, and the breadth and depth of her knowledge is unparalleled.

—Josh Wills, Software Engineer at WeaveGrid and former Director of Data Engineering, Slack

This is the book I wish I had read when I started as an ML engineer.

—Shreya Shankar, MLOps PhD Student

Designing Machine Learning Systems is a welcome addition to the field of applied machine learning. The book provides a detailed guide for people building end-to-end machine learning systems. Chip Huyen writes from her extensive, hands-on experience building real-world machine learning applications.

—Brian Spiering, Data Science Instructor at Metis

Chip is truly a world-class expert on machine learning systems, as well as a brilliant writer. Both are evident in this book, which is a fantastic resource for anyone looking to learn about this topic.

—Andrey Kurenkov, PhD Candidate at the Stanford AI Lab

Chip Huyen has produced an important addition to the canon of machine learning literature—one that is deeply literate in ML fundamentals, but has a much more concrete and practical approach than most. The focus on business requirements alone is uncommon and valuable. This book will resonate with engineers getting started with ML and with others in any part of the organization trying to understand how ML works.

—Todd Underwood, Senior Engineering Director for ML SRE, Google, and Coauthor of *Reliable Machine Learning*

Designing Machine Learning Systems

An Iterative Process for Production-Ready Applications

Chip Huyen

Designing Machine Learning Systems

by Chip Huyen

Copyright © 2022 Huyen Thi Khanh Nguyen. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

- Acquisitions Editor: Nicole Butterfield
- Development Editor: Jill Leonard
- Production Editor: Gregory Hyman
- Copyeditor: nSight, Inc.
- Proofreader: Piper Editorial Consulting, LLC
- Indexer: nSight, Inc.
- Interior Designer: David Futato
- Cover Designer: Karen Montgomery
- Illustrator: Kate Dullea
- May 2022: First Edition

Revision History for the First Edition

- 2022-05-17: First Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781098107963> for release details.

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Designing Machine Learning Systems*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the author, and do not represent the publisher's views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-098-10796-3

[LSI]

Preface

Ever since the first machine learning course I taught at Stanford in 2017, many people have asked me for advice on how to deploy ML models at their organizations. These questions can be generic, such as “What model should I use?” “How often should I retrain my model?” “How can I detect data distribution shifts?” “How do I ensure that the features used during training are consistent with the features used during inference?”

These questions can also be specific, such as “I’m convinced that switching from batch prediction to online prediction will give our model a performance boost, but how do I convince my manager to let me do so?” or “I’m the most senior data scientist at my company and I’ve recently been tasked with setting up our first machine learning platform; where do I start?”

My short answer to all these questions is always: “It depends.” My long answers often involve hours of discussion to understand where the questioner comes from, what they’re actually trying to achieve, and the pros and cons of different approaches for their specific use case.

ML systems are both complex and unique. They are complex because they consist of many different components (ML algorithms, data, business logic, evaluation metrics, underlying infrastructure, etc.) and involve many different stakeholders (data scientists, ML engineers, business leaders, users, even society at large). ML systems are unique because they are data dependent, and data varies wildly from one use case to the next.

For example, two companies might be in the same domain (ecommerce) and have the same problem that they want ML to solve (recommender system), but their resulting ML systems can have different model architecture, use different sets of features, be evaluated on different metrics, and bring different returns on investment.

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

Many blog posts and tutorials on ML production focus on answering one specific question. While the focus helps get the point across, they can create the impression that it's possible to consider each of these questions in isolation. In reality, changes in one component will likely affect other components. Therefore, it's necessary to consider the system as a whole while attempting to make any design decision.

This book takes a holistic approach to ML systems. It takes into account different components of the system and the objectives of different stakeholders involved. The content in this book is illustrated using actual case studies, many of which I've personally worked on, backed by ample references, and reviewed by ML practitioners in both academia and industry. Sections that require in-depth knowledge of a certain topic—e.g., batch processing versus stream processing, infrastructure for storage and compute, and responsible AI—are further reviewed by experts whose work focuses on that one topic. In other words, this book is an attempt to give nuanced answers to the aforementioned questions and more.

When I first wrote the lecture notes that laid the foundation for this book, I thought I wrote them for my students to prepare them for the demands of their future jobs as data scientists and ML engineers. However, I soon realized that I also learned tremendously through the process. The initial drafts I shared with early readers sparked many conversations that tested my assumptions, forced me to consider different perspectives, and introduced me to new problems and new approaches.

I hope that this learning process will continue for me now that the book is in your hand, as you have experiences and perspectives that are unique to you. Please feel free to share with me any feedback you might have for this book, via the [MLOps Discord server](#) that I run (where you can also find other readers of this book), [Twitter](#), [LinkedIn](#), or other channels that you can find on my [website](#).

Who This Book Is For

This book is for anyone who wants to leverage ML to solve real-world problems. ML in this book refers to both deep learning and classical algorithms, with a leaning toward ML systems at scale, such as those seen at medium to large enterprises and fast-growing startups. Systems at a smaller scale tend to be less complex and might benefit less from the comprehensive approach laid out in this book.

Because my background is engineering, the language of this book is geared toward engineers, including ML engineers, data scientists, data engineers, ML platform engineers, and engineering managers. You might be able to relate to one of the following scenarios:

- You have been given a business problem and a lot of raw data. You want to engineer this data and choose the right metrics to solve this problem.
- Your initial models perform well in offline experiments and you want to deploy them.

- You have little feedback on how your models are performing after your models are deployed, and you want to figure out a way to quickly detect, debug, and address any issue your models might run into in production.
- The process of developing, evaluating, deploying, and updating models for your team has been mostly manual, slow, and error-prone. You want to automate and improve this process.
- Each ML use case in your organization has been deployed using its own workflow, and you want to lay down the foundation (e.g., model store, feature store, monitoring tools) that can be shared and reused across use cases.
- You're worried that there might be biases in your ML systems and you want to make your systems responsible!

You can also benefit from the book if you belong to one of the following groups:

- Tool developers who want to identify underserved areas in ML production and figure out how to position your tools in the ecosystem.
- Individuals looking for ML-related roles in the industry.
- Technical and business leaders who are considering adopting ML solutions to improve your products and/or business processes. Readers without strong technical backgrounds might benefit the most from Chapters 1, 2, and 11.

What This Book Is Not

This book is not an introduction to ML. There are many books, courses, and resources available for ML theories, and therefore, this book shies away from these concepts to focus on the practical aspects of ML. To be specific, the book assumes that readers have a basic understanding of the following topics:

- *ML models* such as clustering, logistic regression, decision trees, collaborative filtering, and various neural network architectures including feed-forward, recurrent, convolutional, and transformer
- *ML techniques* such as supervised versus unsupervised, gradient descent, objective/loss function, regularization, generalization, and hyperparameter tuning
- *Metrics* such as accuracy, F1, precision, recall, ROC, mean squared error, and log-likelihood
- *Statistical concepts* such as variance, probability, and normal/long-tail distribution
- *Common ML tasks* such as language modeling, anomaly detection, object classification, and machine translation

You don't have to know these topics inside out—for concepts whose exact definitions can take some effort to remember, e.g., F1 score, we include short notes as references—but you should have a rough sense of what they mean going in.

While this book mentions current tools to illustrate certain concepts and solutions, it's not a tutorial book. Technologies evolve over time. Tools go in and out of style quickly, but fundamental approaches to problem solving should last a bit longer. This book provides a framework for you to evaluate the tool that works best for your use cases. When there's a tool you want to use, it's usually straightforward to find tutorials for it online. As a result, this book has few code snippets and instead focuses on providing a lot of discussion around trade-offs, pros and cons, and concrete examples.

Navigating This Book

The chapters in this book are organized to reflect the problems data scientists might encounter as they progress through the lifecycle of an ML project. The first two chapters lay down the groundwork to set an ML project up for success, starting from the most basic question: does your project need ML? It also covers choosing the objectives for your project and how to frame your problem in a way that makes for simpler solutions. If you're already familiar with these considerations and impatient to get to the technical solutions, feel free to skip the first two chapters.

Chapters 4 to 6 cover the pre-deployment phase of an ML project: from creating the training data and engineering features to developing and evaluating your models in a development environment. This is the phase where expertise in both ML and the problem domain are especially needed.

Chapters 7 to 9 cover the deployment and post-deployment phase of an ML project. We'll learn through a story many readers might be able to relate to that having a model deployed isn't the end of the deployment process. The deployed model will need to be monitored and continually updated to changing environments and business requirements.

Chapters 3 and 10 focus on the infrastructure needed to enable stakeholders from different backgrounds to work together to deliver successful ML systems. Chapter 3 focuses on data systems, whereas Chapter 10 focuses on compute infrastructure and ML platforms. I debated for a long time on how deep to go into data systems and where to introduce it in the book. Data systems, including databases, data formats, data movements, and data processing engines, tend to be sparsely covered in ML coursework, and therefore many data scientists might think of them as low level or irrelevant. After consulting with many of my colleagues, I decided that because ML systems depend on data, covering the basics of data systems early will help us get on the same page to discuss data matters in the rest of the book.

While we cover many technical aspects of an ML system in this book, ML systems are built by people, for people, and can have outsized impact on the life of many. It'd be remiss to write a book on ML production without a chapter on the human side of it, which is the focus of Chapter 11, the last chapter.

Note that “data scientist” is a role that has evolved a lot in the last few years, and there have been many discussions to determine what this role should entail—we’ll go into some of these discussions in [Chapter 10](#). In this book, we use “data scientist” as an umbrella term to include anyone who works developing and deploying ML models, including people whose job titles might be ML engineers, data engineers, data analysts, etc.

GitHub Repository and Community

This book is accompanied by a [GitHub repository](#) that contains:

- A review of basic ML concepts
- A list of references used in this book and other advanced, updated resources
- Code snippets used in this book
- A list of tools you can use for certain problems you might encounter in your workflows

I also run a [Discord server on MLOps](#) where you’re encouraged to discuss and ask questions about the book.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

NOTE

This element signifies a general note.

WARNING

This element indicates a warning or caution.

Using Code Examples

As mentioned, supplemental material (code examples, exercises, etc.) is available for download at <https://oreil.ly/designing-machine-learning-systems-code>.

If you have a technical question or a problem using the code examples, please send email to bookquestions@oreilly.com.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: “*Designing Machine Learning Systems* by Chip Huyen (O'Reilly). Copyright 2022 Huyen Thi Khanh Nguyen, 978-1-098-10796-3.”

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

O'Reilly Online Learning

NOTE

For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit <https://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

- O'Reilly Media, Inc.

- 1005 Gravenstein Highway North
- Sebastopol, CA 95472
- 800-998-9938 (in the United States or Canada)
- 707-829-0515 (international or local)
- 707-829-0104 (fax)

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at <https://oreil.ly/designing-machine-learning-systems>.

Email bookquestions@oreilly.com to comment or ask technical questions about this book.

For news and information about our books and courses, visit <https://oreilly.com>.

Find us on LinkedIn: <https://linkedin.com/company/oreilly-media>

Follow us on Twitter: <https://twitter.com/oreillymedia>

Watch us on YouTube: <https://youtube.com/oreillymedia>

Acknowledgments

This book took two years to write, and many more years beforehand to prepare. Looking back, I'm equally amazed and grateful for the enormous amount of help I received in writing this book. I tried my best to include the names of everyone who has helped me here, but due to the inherent faultiness of human memory, I undoubtedly neglected to mention many. If I forgot to include your name, please know that it wasn't because I don't appreciate your contribution and please kindly remind me so that I can rectify as soon as possible!

First and foremost, I'd like to thank the course staff who helped me develop the course and materials this book was based on: Michael Cooper, Xi Yin, Chloe He, Kinbert Chou, Megan Leszczynski, Karan Goel, and Michele Catasta. I'd like to thank my professors, Christopher Ré and Mehran Sahami, without whom the course wouldn't exist in the first place.

I'd like to thank a long list of reviewers who not only gave encouragement but also improved the book by many orders of magnitude: Eugene Yan, Josh Wills, Han-chung Lee, Thomas Dietterich, Irene Tematelewo, Goku Mohandas, Jacopo Tagliabue, Andrey Kurenkov, Zach Nussbaum, Jay Chia, Laurens Geffert, Brian Spiering, Erin Ledell, Rosanne Liu, Chin Ling, Shreya Shankar, and Sara Hooker.

I'd like to thank all the readers who read the early release version of the book and gave me ideas on how to improve the book, including Charles Frye, Xintong Yu, Jordan Zhang, Jonathon Belotti, and Cynthia Yu.

Of course, the book wouldn't have been possible with the team at O'Reilly, especially my development editor, Jill Leonard, and my production editors, Kristen Brown, Sharon Tripp, and Gregory Hyman. I'd like to thank Laurence Moroney, Hannes Hapke, and Rebecca Novack, who helped me get this book from an idea to a proposal.

This book, after all, is an accumulation of invaluable lessons I learned throughout my career to date. I owe these lessons to my extremely competent and patient coworkers and former coworkers at Claypot AI, Primer AI, Netflix, NVIDIA, and Snorkel AI. Every person I've worked with has taught me something new about bringing ML into the world.

A special thanks to my cofounder Zhenzhong Xu for putting out the fires at our startup and allowing me to spend time on this book. Thank you, Luke, for always being so supportive of everything that I want to do, no matter how ambitious it is.

Chapter 1. Overview of Machine Learning Systems

In November 2016, Google announced that it had incorporated its multilingual neural machine translation system into Google Translate, marking one of the first success stories of deep artificial neural networks in production at scale.¹ According to Google, with this update, the quality of translation improved more in a single leap than they had seen in the previous 10 years combined.

This success of deep learning renewed the interest in machine learning (ML) at large. Since then, more and more companies have turned toward ML for solutions to their most challenging problems. In just five years, ML has found its way into almost every aspect of our lives: how we access information, how we communicate, how we work, how we find love. The spread of ML has been so rapid that it's already hard to imagine life without it. Yet there are still many more use cases for ML waiting to be explored in fields such as health care, transportation, farming, and even in helping us understand the universe.²

Many people, when they hear “machine learning system,” think of just the ML algorithms being used such as logistic regression or different types of neural networks. However, the algorithm is only a small part of an ML system in production. The system also includes the business requirements that gave birth to the ML project in the first place, the interface where users and developers interact with your system, the data stack, and the logic for developing, monitoring, and updating your models, as well as the infrastructure that enables the delivery of that logic. **Figure 1-1** shows you the different components of an ML system and in which chapters of this book they will be covered.

THE RELATIONSHIP BETWEEN MLOPS AND ML SYSTEMS DESIGN

Ops in MLOps comes from DevOps, short for Developments and Operations. To operationalize something means to bring it into production, which includes deploying, monitoring, and maintaining it. MLOps is a set of tools and best practices for bringing ML into production.

ML systems design takes a system approach to MLOps, which means that it considers an ML system holistically to ensure that all the components and their stakeholders can work together to satisfy the specified objectives and requirements.

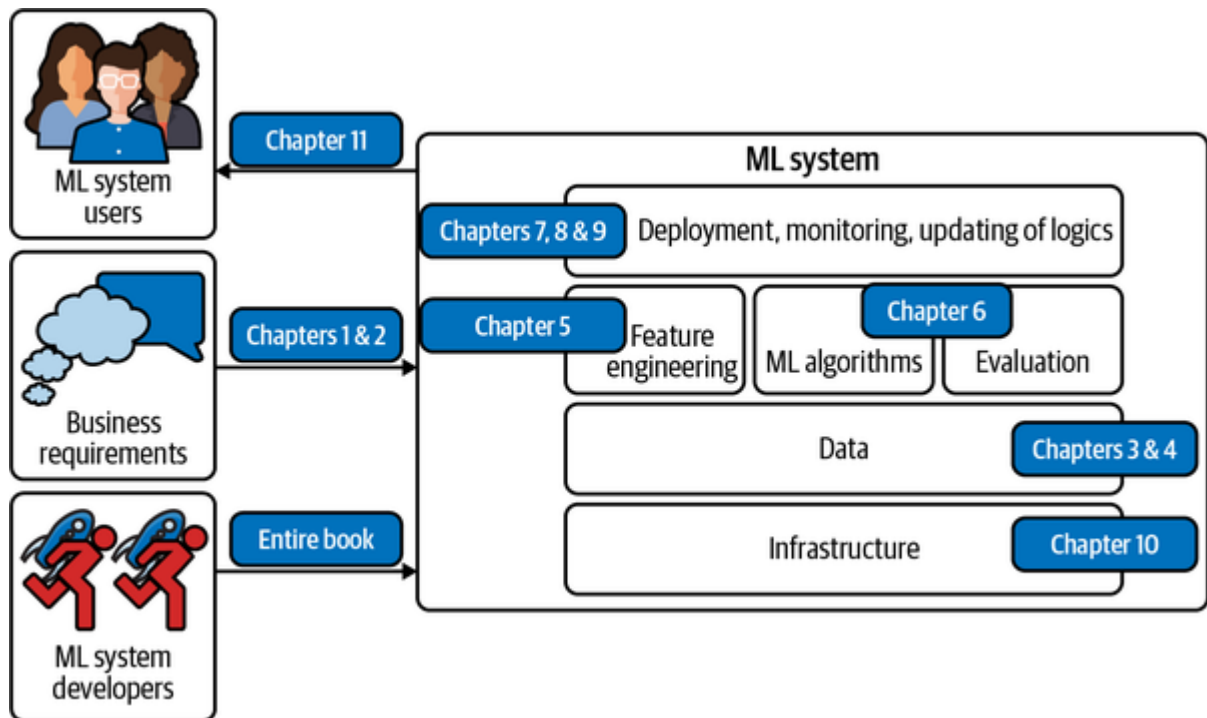


Figure 1-1. Different components of an ML system. “ML algorithms” is usually what people think of when they say machine learning, but it’s only a small part of the entire system.

There are many excellent books about various ML algorithms. This book doesn’t cover any specific algorithms in detail but rather helps readers understand the entire ML system as a whole. In other words, this book’s goal is to provide you with a framework to develop a solution that best works for your problem, regardless of which algorithm you might end up using. Algorithms might become outdated quickly as new algorithms are constantly being developed, but the framework proposed in this book should still work with new algorithms.

The first chapter of the book aims to give you an overview of what it takes to bring an ML model to production. Before discussing how to develop an ML system, it’s important to ask a fundamental question of when and when not to use ML. We’ll cover some of the popular use cases of ML to illustrate this point.

After the use cases, we’ll move on to the challenges of deploying ML systems, and we’ll do so by comparing ML in production to ML in research as well as to traditional software. If you’ve been in the trenches of developing applied ML systems, you might already be familiar with what’s written in this

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

chapter. However, if you have only had experience with ML in an academic setting, this chapter will give an honest view of ML in the real world and set your first application up for success.

When to Use Machine Learning

As its adoption in the industry quickly grows, ML has proven to be a powerful tool for a wide range of problems. Despite an incredible amount of excitement and hype generated by people both inside and outside the field, ML is not a magic tool that can solve all problems. Even for problems that ML can solve, ML solutions might not be the optimal solutions. Before starting an ML project, you might want to ask whether ML is necessary or cost-effective.³

To understand what ML can do, let's examine what ML solutions generally do:

Machine learning is an approach to (1) learn (2) complex patterns from (3) existing data and use these patterns to make (4) predictions on (5) unseen data.

We'll look at each of the italicized keyphrases in the above framing to understand its implications to the problems ML can solve:

1. Learn: the system has the capacity to learn

A relational database isn't an ML system because it doesn't have the capacity to learn. You can explicitly state the relationship between two columns in a relational database, but it's unlikely to have the capacity to figure out the relationship between these two columns by itself.

For an ML system to learn, there must be something for it to learn from. In most cases, ML systems learn from data. In supervised learning, based on example input and output pairs, ML systems learn how to generate outputs for arbitrary inputs. For example, if you want to build an ML system to learn to predict the rental price for Airbnb listings, you need to provide a dataset where each input is a listing with relevant characteristics (square footage, number of rooms, neighborhood, amenities, rating of that listing, etc.) and the associated output is the rental price of that listing. Once learned, this ML system should be able to predict the price of a new listing given its characteristics.

2. Complex patterns: there are patterns to learn, and they are complex

ML solutions are only useful when there are patterns to learn. Sane people don't invest money into building an ML system to predict the next outcome of a fair die because there's no pattern in how these outcomes are generated.⁴ However, there are patterns in how stocks are priced, and therefore companies have invested billions of dollars in building ML systems to learn those patterns.

Whether a pattern exists might not be obvious, or if patterns exist, your dataset or ML algorithms might not be sufficient to capture them. For example, there might be a pattern in how Elon Musk's tweets affect cryptocurrency prices. However, you wouldn't know until you've rigorously trained and evaluated your ML models on his tweets. Even if all your models fail to make reasonable predictions of cryptocurrency prices, it doesn't mean there's no pattern.

Consider a website like Airbnb with a lot of house listings; each listing comes with a zip code. If you want to sort listings into the states they are located in, you wouldn't need an ML system. Since the pattern is simple—each zip code corresponds to a known state—you can just use a lookup table.

The relationship between a rental price and all its characteristics follows a much more complex pattern, which would be very challenging to manually specify. ML is a good solution for this. Instead of telling your system how to calculate the price from a list of characteristics, you can provide prices and characteristics, and let your ML system figure out the pattern. The difference between ML solutions and the lookup table solution as well as general traditional software solutions is shown in **Figure 1-2**. For this reason, ML is also called Software 2.0.⁵

ML has been very successful with tasks with complex patterns such as object detection and speech recognition. What is complex to machines is different from what is complex to humans. Many tasks that are hard for humans to do are easy for machines—for example, raising a number to the power of 10. On the other hand, many tasks that are easy for humans can be hard for machines—for example, deciding whether there's a cat in a picture.

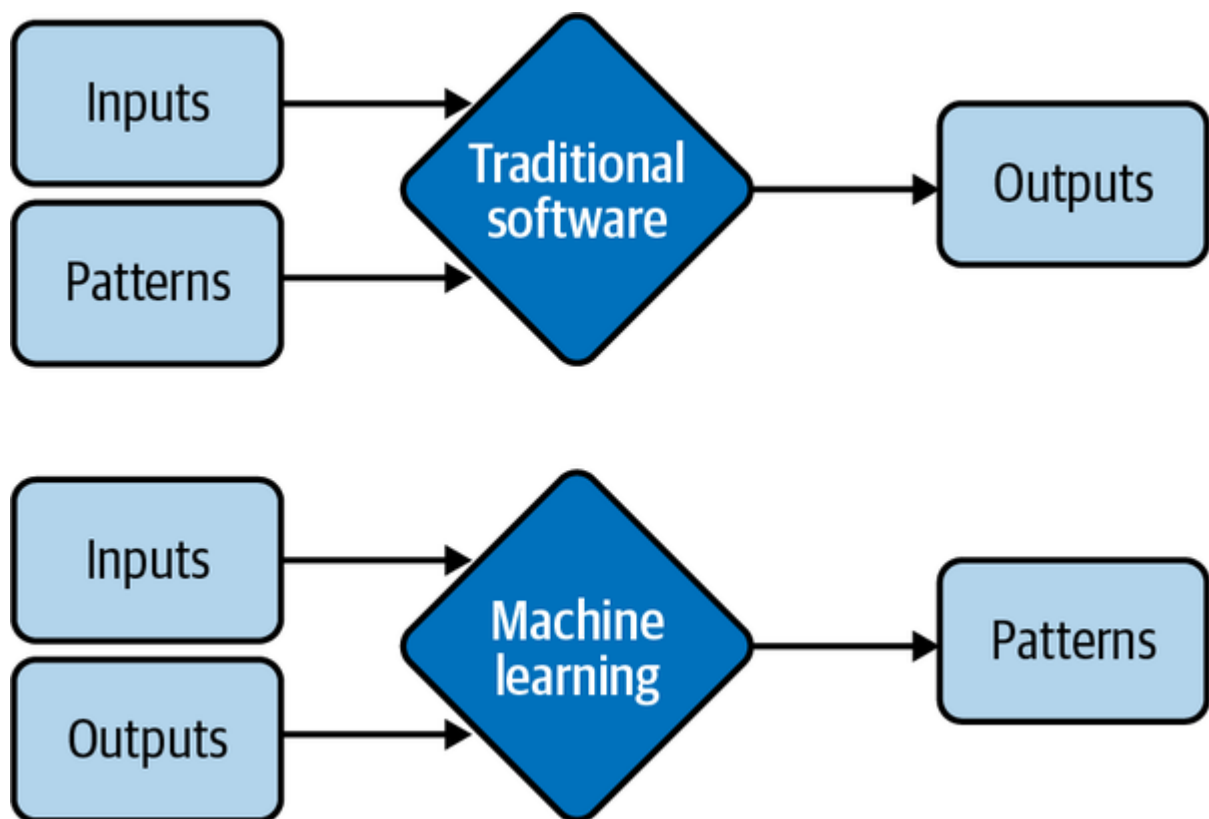


Figure 1-2. Instead of requiring hand-specified patterns to calculate outputs, ML solutions learn patterns from inputs and outputs

3. Existing data: data is available, or it's possible to collect data

Because ML learns from data, there must be data for it to learn from. It's amusing to think about building a model to predict how much tax a person should pay a year, but it's not possible unless you have access to tax and income data of a large population.

In the **zero-shot learning** (sometimes known as zero-data learning) context, it's possible for an ML system to make good predictions for a task without having been trained on data for that task. However, this ML system was previously trained on data for other tasks, often related to the task in consideration. So even though the system doesn't require data for the task at hand to learn from, it still requires data to learn.

It's also possible to launch an ML system without data. For example, in the context of continual learning, ML models can be deployed without having been trained on any data, but they will learn from incoming data in production.⁶ However, serving insufficiently trained models to users comes with certain risks, such as poor customer experience.

Without data and without continual learning, many companies follow a “fake-it-til-you make it” approach: launching a product that serves predictions made by humans, instead of ML models, with the hope of using the generated data to train ML models later.

4. Predictions: it's a predictive problem

ML models make predictions, so they can only solve problems that require predictive answers. ML can be especially appealing when you can benefit from a large quantity of cheap but approximate predictions. In English, “predict” means “estimate a value in the future.” For example, what will the weather be like tomorrow? Who will win the Super Bowl this year? What movie will a user want to watch next?

As predictive machines (e.g., ML models) are becoming more effective, more and more problems are being reframed as predictive problems. Whatever question you might have, you can always frame it as: “What would the answer to this question be?” regardless of whether this question is about something in the future, the present, or even the past.

Compute-intensive problems are one class of problems that have been very successfully reframed as predictive. Instead of computing the exact outcome of a process, which might be even more computationally costly and time-consuming than ML, you can frame the problem as: “What would the outcome of this process look like?” and approximate it using an ML model. The output will be an approximation of the exact output, but often, it's good enough. You can see a lot of it in graphic renderings, such as image denoising and screen-space shading.⁷

5. Unseen data: unseen data shares patterns with the training data

The patterns your model learns from existing data are only useful if unseen data also share these patterns. A model to predict whether an app will get downloaded on Christmas 2020 won't perform very well if it's trained on data from 2008, when the most popular app on the App Store was Koi Pond. What's Koi Pond? Exactly.

In technical terms, it means your unseen data and training data should come from similar distributions. You might ask: “If the data is unseen, how do we know what distribution it comes

from?” We don’t, but we can make assumptions—such as we can assume that users’ behaviors tomorrow won’t be too different from users’ behaviors today—and hope that our assumptions hold. If they don’t, we’ll have a model that performs poorly, which we might be able to find out with monitoring, as covered in [Chapter 8](#), and test in production, as covered in [Chapter 9](#).

Due to the way most ML algorithms today learn, ML solutions will especially shine if your problem has these additional following characteristics:

6. It’s repetitive

Humans are great at few-shot learning: you can show kids a few pictures of cats and most of them will recognize a cat the next time they see one. Despite exciting progress in few-shot learning research, most ML algorithms still require many examples to learn a pattern. When a task is repetitive, each pattern is repeated multiple times, which makes it easier for machines to learn it.

7. The cost of wrong predictions is cheap

Unless your ML model’s performance is 100% all the time, which is highly unlikely for any meaningful tasks, your model is going to make mistakes. ML is especially suitable when the cost of a wrong prediction is low. For example, one of the biggest use cases of ML today is in recommender systems because with recommender systems, a bad recommendation is usually forgiving—the user just won’t click on the recommendation.

If one prediction mistake can have catastrophic consequences, ML might still be a suitable solution if, on average, the benefits of correct predictions outweigh the cost of wrong predictions. Developing self-driving cars is challenging because an algorithmic mistake can lead to death. However, many companies still want to develop self-driving cars because they have the potential to save many lives once self-driving cars are statistically safer than human drivers.

8. It’s at scale

ML solutions often require nontrivial up-front investment on data, compute, infrastructure, and talent, so it’d make sense if we can use these solutions a lot.

“At scale” means different things for different tasks, but, in general, it means making a lot of predictions. Examples include sorting through millions of emails a year or predicting which departments thousands of support tickets should be routed to a day.

A problem might appear to be a singular prediction, but it’s actually a series of predictions. For example, a model that predicts who will win a US presidential election seems like it only makes one prediction every four years, but it might actually be making a prediction every hour or even more frequently because that prediction has to be continually updated to incorporate new information.

Having a problem at scale also means that there's a lot of data for you to collect, which is useful for training ML models.

9. The patterns are constantly changing

Cultures change. Tastes change. Technologies change. What's trendy today might be old news tomorrow. Consider the task of email spam classification. Today an indication of a spam email is a Nigerian prince, but tomorrow it might be a distraught Vietnamese writer.

If your problem involves one or more constantly changing patterns, hardcoded solutions such as handwritten rules can become outdated quickly. Figuring how your problem has changed so that you can update your handwritten rules accordingly can be too expensive or impossible. Because ML learns from data, you can update your ML model with new data without having to figure out how the data has changed. It's also possible to set up your system to adapt to the changing data distributions, an approach we'll discuss in the section **"Continual Learning"**.

The list of use cases can go on and on, and it'll grow even longer as ML adoption matures in the industry. Even though ML can solve a subset of problems very well, it can't solve and/or shouldn't be used for a lot of problems. Most of today's ML algorithms shouldn't be used under any of the following conditions:

- It's unethical. We'll go over one case study where the use of ML algorithms can be argued as unethical in the section **"Case study I: Automated grader's biases"**.
- Simpler solutions do the trick. In **Chapter 6**, we'll cover the four phases of ML model development where the first phase should be non-ML solutions.
- It's not cost-effective.

However, even if ML can't solve your problem, it might be possible to break your problem into smaller components, and use ML to solve some of them. For example, if you can't build a chatbot to answer all your customers' queries, it might be possible to build an ML model to predict whether a query matches one of the frequently asked questions. If yes, direct the customer to the answer. If not, direct them to customer service.

I'd also want to caution against dismissing a new technology because it's not as cost-effective as the existing technologies at the moment. Most technological advances are incremental. A type of technology might not be efficient now, but it might be over time with more investments. If you wait for the technology to prove its worth to the rest of the industry before jumping in, you might end up years or decades behind your competitors.

Machine Learning Use Cases

ML has found increasing usage in both enterprise and consumer applications. Since the mid-2010s, there has been an explosion of applications that leverage ML to deliver superior or previously impossible services to consumers.

With the explosion of information and services, it would have been very challenging for us to find what we want without the help of ML, manifested in either a *search engine* or a *recommender system*. When you visit a website like Amazon or Netflix, you're recommended items that are predicted to best match your taste. If you don't like any of your recommendations, you might want to search for specific items, and your search results are likely powered by ML.

If you have a smartphone, ML is likely already assisting you in many of your daily activities. Typing on your phone is made easier with *predictive typing*, an ML system that gives you suggestions on what you might want to say next. An ML system might run in your photo editing app to suggest how best to enhance your photos. You might authenticate your phone using your fingerprint or your face, which requires an ML system to predict whether a fingerprint or a face matches yours.

The ML use case that drew me into the field was *machine translation*, automatically translating from one language to another. It has the potential to allow people from different cultures to communicate with each other, erasing the language barrier. My parents don't speak English, but thanks to Google Translate, now they can read my writing and talk to my friends who don't speak Vietnamese.

ML is increasingly present in our homes with smart personal assistants such as Alexa and Google Assistant. Smart security cameras can let you know when your pets leave home or if you have an uninvited guest. A friend of mine was worried about his aging mother living by herself—if she falls, no one is there to help her get up—so he relied on an at-home health monitoring system that predicts whether someone has fallen in the house.

This document was truncated here because it was created in the Evaluation Mode.